Bioinformatics, 2026, pp. 1–5

Application Note

RIsearch and siOFF: An integrated, high-performance framework for RNA–RNA interaction and siRNA off-target assessment

Stefano Roncelli [1], Lorenzo Favaro [1], Christian Anthon [1] and Jan Gorodkin [1,*]

[1]Center for non-coding RNA in Technology and Health, Section for Health Data Science and AI, Department of Public Health, University of Copenhagen, Thorvaldsensvej 57, 1871, Frederiksberg, Denmark

*Corresponding author. gorodkin@rth.dk

Abstract

Motivation: A substantial part of RNA function comes from its interactions with other RNAs. Given the sheer scale of the search space (genome/transcriptome), there is high demand for fast RNA interaction search software that should also be user-friendly and easy to integrate with other tools.

Results: Here, we introduce new versions of RIsearch and its downstream siRNA off-target assessment pipeline. Both tools now have substantially improved usability, interoperability, and installation procedures. RIsearch (v3) is a Rust reimplementation of the RIsearch2 search algorithm, verified against the reference; it reworks the original command-line interface, adds Python bindings, and improves performance. siOFF modernises the previous siRNA off-target pipeline through a redesigned CLI, improved distribution and faster execution.

Availability and implementation: RIsearch and siOFF are released under the Business Source License 1.1 at github.com/RTH-tools/risearch and github.com/RTH-tools/siOFF. Both software packages are available on PyPI as `risearch` and `sioff`; the Rust crate is also indexed on Crates.io as `risearch`.

Contact: Jan Gorodkin, gorodkin@rth.dk

Keywords RNA-RNA interactions, siRNA off-target assessment, Rust, Python

Introduction and motivation

Intermolecular RNA interactions with both RNA and DNA underpin a wide range of regulatory mechanisms, and demand is increasing for predicting them at genome- or transcriptome-wide scale. While RNA–RNA interactions continue to receive increasing attention, recent work has linked RNA–DNA interactions to cell type-specific genome regulation (Lambolez et al., 2026). Even when studying and using endogenous or synthetic small interfering RNAs (siRNAs), unaccounted off-target interactions can confound RNAi knockdown experiments and limit therapeutic applications.

To computationally search for such off-targets, RIsearch2 (Alkan et al., 2017) was designed for large-scale screens of near-complementary RNA–RNA interactions. Because it ignores intramolecular interactions, it is particularly suitable for probing the off-target landscape of small RNAs, such as siRNAs. It combines a suffix-array index of the target with a seed-and-extend dynamic-programming step driven by a simplified nearest-neighbour energy model (Wenzel et al., 2012). The companion siRNA off-target pipeline turns these predictions into per-transcript binding probabilities and a transcriptome-wide off-targeting potential through a partition-function model that integrates transcript abundance and binding-site accessibility.

With the development of modern software ecosystems, new opportunities have emerged to improve how RIsearch2 and its pipeline were originally distributed and used. Here, we introduce new versions of RIsearch2 and the siRNA pipeline, redesigned for improved usability, interoperability with the latest frameworks, and easier installation. RIsearch (v3) is a faster, memory-safe Rust reimplementation of the core search algorithm, with a redesigned command-line interface and PyO3 Python bindings. The siRNA Off-Target pipeline, siOFF, is a rewrite of the previous siRNA off-target pipeline that consumes the RIsearch (v3) bindings directly (Fig. 1).

A limitation of the original RIsearch2 package is its distribution and installation: it is provided as C source code that each user must compile locally. RIsearch (v3) improves on these aspects while meeting three essential requirements at once:

1. retaining the genome-scale throughput that keeps whole-transcriptome screens practical;
2. installing as a prebuilt package from a standard registry, with no manual build;

Received: Date Month Year. **Revised received:** Date Month Year. **Accepted:** Date Month Year

© The Author(s) 2026. Copyright and licence statements to be updated by the publisher during production.

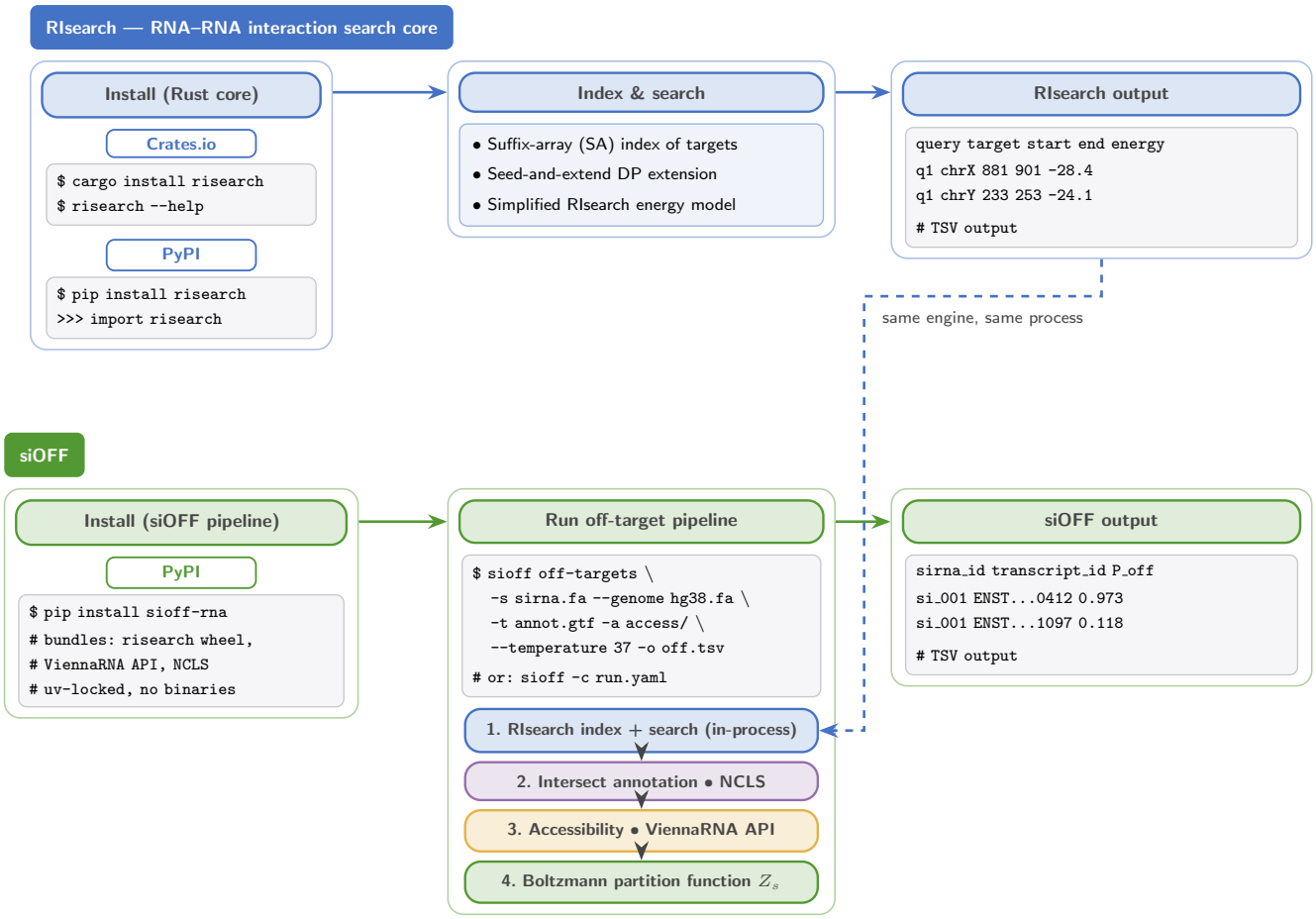


Figure 1 Installation and execution of RIsearch and the siOFF pipeline. (Top, blue) RIsearch is a Rust reimplementation of the RIsearch2 search algorithm, installable via `cargo install risearch` (Crates.io) or as a Python wheel via `pip install risearch` (PyPI). The core builds a suffix-array index of target sequences and runs a seed-and-extend dynamic-programming search using the simplified RIsearch energy model, producing interaction hits as a TSV file. **(Bottom, green) siOFF** (siRNA Off-Target pipeline) is a timely Python 3 rewrite installable with a single `pip install sioff-rna`, which bundles the RIsearch dependencies, the ViennaRNA 2.7.2 Python API, and an in-memory nested containment list (NCLS) interval index with no external binaries required. siOFF orchestrates four stages: (*i*) RIsearch index and search invoked natively via PyO3 bindings; (*ii*) intersection of predicted binding sites with a transcriptome annotation using an NCLS index; (*iii*) per-site RNA accessibility computed through the ViennaRNA Python API; and (*iv*) a per-siRNA Boltzmann partition function yielding off-target probabilities, written to a TSV output file. The dashed arrow indicates that siOFF calls RIsearch within the same process via the Python bindings, with no subprocess or intermediate files.

3. running as a memory-safe Python library, so off-target searches integrate seamlessly with downstream analyses.

While each of 1)–3) is routine in itself, no single established language meets all three. C reaches that throughput, but has no standard package registry for prebuilt distribution and cannot guarantee memory safety at a library boundary. Pure Python, conversely, cannot reach that throughput because its interpreter runs the seed-search and dynamic-programming loops far slower than compiled native code, even though it distributes and embeds easily. Rust, however, meets all three: it compiles to fast native code, installs from standard package registries, and exposes the algorithm through a memory-safe Python library. These capabilities are why we rebuilt RIsearch in Rust.

RIsearch reimplementation

RIsearch (v3) is a complete Rust reimplementation of the RIsearch2 seed-and-extend algorithm: targets are concatenated and indexed in a suffix array, and each query is scanned against the index for near-complementary seeds. Seeds are then extended in both directions using Gotoh's three-state affine-gap recurrence (Gotoh, 1982), scoring stacked pairs from a nearest-neighbour dinucleotide model that penalises bulges and internal loops. Most of the codebase is written in safe Rust to guarantee memory safety, with unchecked memory access confined to the performance-critical inner loops of seed search and extension and to the zero-copy loading of the on-disk index. The suffix array is now constructed with libsais (Grebnov, 2021), a linear-time algorithm that runs in parallel across cores, replacing the single-threaded libdivsufsort (Mori, 2015) construction used by RIsearch2. Finally, we regenerated the energy parameter tables from ViennaRNA duplex energies, so the calculated hybridisation energies differ slightly from those of RIsearch2.

Seamless installation

Compiling RIsearch2 from source requires a local C toolchain and leaves compiler flags and dependency versions to each machine.

RIsearch (v3) requires no manual build: it installs with a single command from the standard Python and Rust package registries, PyPI and Crates.io, or as a prebuilt binary from each GitHub release.

Improved usability

While RIsearch2 often returns multiple redundant alignments for a single binding site when adjacent seeds extend into the same duplex, RIsearch (v3) deduplicates these by default: seed extensions that converge on identical query and target coordinates are collapsed to the single alignment with the most favourable free energy. Distinct suboptimal binding sites elsewhere on the transcript are retained, and users can disable collapsing via a flag to inspect every uncollapsed seed extension. RIsearch (v3) also enforces strict input and index validation: it auto-detects plain or compressed FASTA/FASTQ inputs, checks sequence characters and duplicate record IDs, and verifies on-disk index version headers and archive integrity, aborting on malformed data with an error naming the offending file, record, or line. Finally, the command-line interface is organised around task-oriented subcommands (`risearch index` and `risearch search`) while still accepting legacy RIsearch2 flags to ease migration of existing scripts.

Enhanced interoperability

At the time of implementation of RIsearch2, API libraries were not mainstream in bioinformatics, so embedding it in a Python analysis meant spawning a subprocess and parsing its text output. The new version of RIsearch instead ships PyO3 bindings in the same wheel: `risearch.index()` builds an on-disk index, `risearch.TargetRegistry.open()` loads it as a reusable memory-mapped handle, and `risearch.search()` returns hits as a Polars DataFrame through the Arrow C stream interface, with no intermediate files and no text parsing:

```
import risearch
risearch.index("targets.fa", "index/")
store = risearch.TargetRegistry.open("index/")
df = risearch.search(queries, store, matrix="t04")
```

Searches run without blocking other Python threads in the same process, and the search parameter set can be chosen per call.

Extensible energy parameters and engine

The energy parameters are stored as tables separate from the search code: the bundled sets are generated from ViennaRNA by a script in the RIsearch repository, and users can load their own table at runtime, so new thermodynamic parameters require no change to the search engine and no knowledge of Rust internals. The engine itself is modular, and its behaviour is verified against independent reference implementations through property-based and metamorphic tests, complemented by mutation testing and bounded model-checking proofs, so contributors can extend it with confidence.

Temperature dependent energy parameters

RIsearch1 and RIsearch2 were released with energy parameters for RNA-RNA interactions at 37 degrees Celsius only. Later versions **?** include parameters for RNA-DNA interactions, at 37 degrees only. For RIsearch3, we re-implemented the energy parameters by computing interaction energies with ViennaRNA for interacting nucleotide duplexes and obtained the stacking energy parameters for the RIsearch model by fitting (see Supplementary section **??** for details). This way, we obtained stacking energy parameters for RNA/RNA, DNA/DNA, RNA/DNA and DNA/RNA query and target interactions. The parameters were derived at 0, 25, 37, 42, and 50 degrees Celsius, and RIsearch3 uses these stacking parameters directly or via linear interpolation to compute interaction energies at input temperatures.

Faster execution time

On an AMD EPYC 9554 server, with both tools searching the same 63 siRNAs against the same chromosome-X transcript set, RIsearch (v3) is 2–3× faster than RIsearch2 (Supplementary Figures S1–S4; Supplementary Results S1.1). The suffix array creation rewrite also yielded about 2× faster execution times (Supplementary Figure S5; Supplementary Results S1.2). Because the on-disk index is now memory-mapped and read without deserialisation, multi-gigabyte genome indexes are available immediately and shared across processes by the operating system, rather than read into memory on every run.

siOFF: A reimplemented siRNA Off-Target assessment pipeline

Alongside the new version of RIsearch, we distribute **siOFF**, a complete reimplementation of the siRNA off-target pipeline introduced by Alkan et al. (2017) (Fig. 1). Beyond porting the workflow from the end-of-life Python 2.7 to Python 3, the rewrite dismantles the concrete obstacles that limited adoption of the original pipeline: an installation that required a hand-compiled RIsearch2 binary and two external executables, a rigid single-script interface with a fixed set of flags, and subprocess-and-file coupling that prevented embedding in larger analyses. The underlying four-stage thermodynamic model—search, annotation intersection, accessibility, and a per-siRNA Boltzmann partition function—is preserved unchanged and described in full in S2.1.

One-command installation

The original pipeline requires a Python 2.7 interpreter, a separately compiled RIsearch2 C binary, and installation of external software `RNAplfold` (ViennaRNA) and `bedtools` (Quinlan and Hall, 2010). siOFF removes the need for external dependency installation: a single `pip install sioff-rna` brings in the RIsearch software as a pre-compiled wheel, the ViennaRNA 2.7.2 Python API (Lorenz et al., 2011) in place of the `RNAplfold` executable, and an in-memory nested containment list (NCLS) (Alekseyenko and Lee, 2007)—an interval index that answers genomic overlap queries directly in Python—in place of `bedtools`. All transitive dependencies are resolved and pinned through a universal `uv` lockfile, so installations are reproducible and free of version conflicts across machines.

Easier to use

The legacy pipeline was a single `argparse` script driven by a fixed set of command-line flags. siOFF retains backwards-compatible flags but adds a YAML configuration mode: every option—inputs, the RIsearch energy model, seed geometry, and energy threshold, the saturation parameters α, γ, and θ, and the output format—can be declared in one file, validated at startup against a typed OmegaConf schema.

A single configuration file captures an entire run, making analyses straightforward to reproduce and to share between collaborators. For large screens, siOFF integrates natively with Slurm: one orchestrator submits the indexing, accessibility, and off-target stages as dependent batch jobs with per-stage resource requests, and a single configuration can fan one screen out across multiple transcriptomes as independent job groups, replacing hand-written submission scripts.

We have now implemented a temperature option (default 37 °C). The temperature propagates to both the ViennaRNA accessibility calculation and the Boltzmann partition function, so that hybridisation and opening energies are always evaluated on a consistent thermodynamic scale—directly replicating the intent of the original RIsearch model (Wenzel et al., 2012) while exposing the folding condition as a first-class parameter (Supplementary Methods S3).

Easy integration

Because the legacy pipeline communicated through intermediate files and subprocesses, embedding it in a larger analysis meant shelling out to a script and parsing its text output. siOFF instead exposes its stages as an importable Python API: the search, intersection, accessibility, and probability services can each be called directly and return native Polars DataFrames, so a user can drop any stage into an existing Python workflow and pass results to other Python-native tools in a single process, with no intermediate files and no text parsing. Based on RIsearch software bindings (Section 2), siOFF thus serves both as a turnkey command-line tool and as a library component of off-target ad hoc analyses.

Enhanced execution time

The pipeline was benchmarked on an AMD EPYC 9554 (64 cores, 32 workers) using subsets of the genome-wide siRNA efficacy library of Huesken et al. (2005) searched against the GRCh38/hg38 reference genome (Schneider et al., 2017) and intersected against an H1299 RNA-seq expression profile (dataset, genome indexing, expression annotation, and benchmark protocol detailed in Supplementary Methods S2.3–S2.6), siOFF completes the intersection, accessibility annotation, probability computation, and output stages 20.1−23.3× faster than the legacy Python 2.7 implementation across batch sizes of 500–2,000 siRNAs (five timed runs per batch size; Supplementary Figure S6). The speed-up comes at the cost of a larger total memory footprint: summed over all 32 workers, siOFF peaks at ∼47–52 GB versus ∼12–17 GB for the legacy pipeline (Supplementary Figure S6; worker model and memory behaviour detailed in Supplementary Methods S2.2).

Separately, the one-time genome-wide accessibility pre-computation—folding the entire hg38 reference at W=80, L=40, u=30 with 64 workers—runs ∼21% faster than the legacy `RNAplfold` script (9.4 h vs. 11.9 h) and writes profiles of the same size (192.6 GB), stored in a layout that the off-target stage memory-maps directly, at the cost of a larger memory footprint (peak proportional set size summed over all processes ∼147 GB vs. ∼18.5 GB; Supplementary Figure S7 and Supplementary Table S1; method detailed in Supplementary Methods S2.7).

Final Remarks

Here, we have exploited the emerging possibilities of new software ecosystems to reimplement RIsearch and the siOFF pipeline, with enhancements in installation, maintenance, integration with other tools and speed. Both tools can be used from the command line and within Python workflows by anyone screening RNA interactions or siRNA off-targets at transcriptome scale. Future work will extend siOFF to the combined off-targeting potential of pooled siRNAs and integrate RIsearch with further analysis frameworks.

Author contributions statement

S.R. reimplemented RIsearch2 in Rust and provided the framework for the siOFF pipeline. L.F. designed and implemented the siOFF pipeline, ran all benchmarks, and drafted the manuscript. C.A. designed the temperature option and tested the software. J.G. supervised the project. All authors commented on, contributed to, and approved the final manuscript.

Funding

This work is supported by the Novo Nordisk Foundation (NNF24OC0087112).

Conflict of interest

None declared.

Code and data availability

The source code for RIsearch and siOFF is available on GitHub (RTH-tools/risearch, RTH-tools/siOFF), and the packages on PyPI (risearch, sioff) and Crates.io (risearch). All distribution channels are summarised at rth.dk/resources/risearch. The benchmarking code and scripts used to produce the results in this paper are deposited on Zenodo (doi:10.5281/zenodo.XXXXXXX).

References

A. V. Alekseyenko and C. J. Lee. Nested Containment List (NCList): a new algorithm for accelerating interval query of genome alignment and interval databases. *Bioinformatics*, 23 (11):1386–1393, 2007. doi: 10.1093/bioinformatics/btl647.

F. Alkan, A. Wenzel, O. Palasca, P. Kerpedjiev, A. F. Rudebeck, P. F. Stadler, I. L. Hofacker, and J. Gorodkin. RIsearch2: suffix array-based large-scale prediction of RNA–RNA interactions and siRNA off-targets. *Nucleic Acids Research*, 45(8):e60, 2017. doi: 10.1093/nar/gkw1325.

O. Gotoh. An improved algorithm for matching biological sequences. *Journal of Molecular Biology*, 162(3):705–708, 1982. doi: 10.1016/0022-2836(82)90398-9.

I. Grebnov. libsais: linear-time suffix array, LCP array and Burrows–Wheeler transform construction. https://github.com/IlyaGrebnov/libsais, 2021. Accessed 2026.

D. Huesken, J. Lange, C. Mickanin, J. Weiler, F. Asselbergs, J. Warner, B. Meloon, S. Engel, A. Rosenberg, D. Cohen, M. Labow, M. Reinhardt, F. Natt, and J. Hall. Design of a genome-wide siRNA library using an artificial neural

network. *Nature Biotechnology*, 23(8):995–1001, 2005. doi: 10.1038/nbt1118.

A. Lambolez, P. Sahlén, W. Kang, X. Shu, J. Severin, R. Pracana, I. Abdelhamid, B. Dhaka, C. Vroland, V. Ranzani, B. Polimeni, M. Koido, A. Vandelli, M. Mintseva, M. Rohaly Medved, K. Yasuzawa, M. Murata, D. Delobel, W. H. Yip, H. Nishiyori-Sueki, S. Takizawa, T. Nobusada, M. Brown, V. Di Gioia, Y. Inaba, S. Kato, C. Parr, K. Kaji, T. Kawashima, T. Kouno, M. Tagami, K. Ozaki, R. Vadalà, F. Marasca, E. Cozzi, R. Krautz, C. Vaagensø, T. Yamazaki, X. Li Wang, Q. Verron, Y. Ichikawa, J.-C. Chang, M. Valentine, H. Einarsson, J. Moody, A. Hasegawa, Z. Liao, K. Tomizuka, A. Zhigulev, M. Gaviraghi, Z. Altan, S. Policicchio, Y. Yu, C. Latini, R. Piccioni, C. Medaglia, A. P. Schnepper, L. Carpen, E. Ricciardelli, M. Bonfanti, R. Bosotti, C. Callierotti, G. M. Scotti, Y.-L. Chen, V. Kumar, A. Karabacak, X. Zhong, A. Naughton, I. Farabella, P. Robbe, N. Pirastu, G. Pascarella, L. P. Tripathi, et al. The human RNA-DNA interactome is cell type-specific and dynamic. *bioRxiv*, 2026. doi: 10.64898/2026.08.25.746868.

R. Lorenz, S. H. Bernhart, C. Höner zu Siederdissen, H. Tafer, C. Flamm, P. F. Stadler, and I. L. Hofacker. ViennaRNA package 2.0. *Algorithms for Molecular Biology*, 6(1):26, 2011. doi: 10.1186/1748-7188-6-26.

Y. Mori. libdivsufsort: a lightweight suffix-array construction library. https://github.com/y-256/libdivsufsort, 2015. Accessed 2026.

A. R. Quinlan and I. M. Hall. BEDTools: a flexible suite of utilities for comparing genomic features. *Bioinformatics*, 26(6):841–842, 2010. doi: 10.1093/bioinformatics/btq033.

V. A. Schneider, T. Graves-Lindsay, K. Howe, N. Bouk, H.-C. Chen, P. A. Kitts, T. D. Murphy, K. D. Pruitt, F. Thibaud-Nissen, D. Albracht, R. S. Fulton, M. Kremitzki, V. Magrini, C. Markovic, S. McGrath, K. M. Steinberg, K. Auger, W. Chow, J. Collins, G. Harden, T. Hubbard, S. Pelan, J. T. Simpson, G. Threadgold, J. Torrance, J. M. Wood, L. Clarke, S. Koren, M. Boitano, P. Peluso, H. Li, C.-S. Chin, A. M. Phillippy, R. Durbin, R. K. Wilson, P. Flicek, E. E. Eichler, and D. M. Church. Evaluation of GRCh38 and de novo haploid genome assemblies demonstrates the enduring quality of the reference assembly. *Genome Research*, 27(5):849–864, 2017. doi: 10.1101/gr.213611.116.

A. Wenzel, E. Akbaşlı, and J. Gorodkin. RIsearch: fast RNA–RNA interaction search using a simplified nearest-neighbor energy model. *Bioinformatics*, 28(21):2738–2746, 2012. doi: 10.1093/bioinformatics/bts519.